Views

View

Customer

id	name
C1	Customer 1
C2	Customer 2
C3	Customer 3
C4	Customer 4

Item

id	name	inventory
I1	Item 1	100
I2	Item 2	200
I3	Item 3	150

Order

id	cust_id	item_id	qty	price
O1	C1	I1	10	$125
O2	C2	I2	15	$365
O3	C3	I1	80	$110
O4	C4	I3	120	$125
O5	C3	I3	40	$150

Stores query, not data.

Fulfillment

ord_id	cust_id	cust_name	item_id	item_name	can_fulfill
O1	C1	Customer 1	I1	Item 1	TRUE
O2	C2	Customer 2	I2	Item 2	TRUE
O3	C3	Customer 3	I1	Item 1	TRUE
O4	C4	Customer 4	I3	Item 3	TRUE
O5	C3	Customer 3	I3	Item 3	FALSE

Using a view

Reading

- SELECT

Writing

- INSERT
- UPDATE

View restrictions:

- Single table
- No computation

View created by a query

```
CREATE VIEW MyView AS
SELECT ...;
```

```
CREATE OR REPLACE VIEW ...
AS
SELECT ...;
```

```
DROP VIEW
...;
```

Some scenarios for views

- Focus on specific data
  - Let multiple users focus on different sets of specific data
- Simplify common data manipulation
- Provide an interface between queries & tables
  - Views provide unchanging interface when tables change
- Import/export data
  - View models file format

Potential problem with views

View doesn’t store data.

View created via query.

Underlying table data can change.

Reusing view may need to recompute query.

Materialized views

Hybrid of view & table.

- Stores results
- Periodically reevaluates query to refresh results
- Can control reevaluation

```
CREATE MATERIALIZED VIEW MyView AS
SELECT ...;
```

Refreshing materialized views

DBMS	Method
PostgreSQL	`REFRESH MATERIALIZED VIEW` *`viewname`*
Oracle	Multiple ways for partial or full refresh
SQL Server	*Indexed views*, instead
MySQL	No equivalent